

OBJECT ORIENTED SYSTEMS (OOS) — 2025

QUESTION 5 COMPLETE DETAILED SOLUTION

Introduction

Question 5 of the 2025 Object Oriented Systems examination belongs to CO5 and focuses on one of the most advanced and industry-relevant areas of software engineering:

Design Patterns

Design patterns are standardized reusable solutions to recurring software design problems  . Instead of repeatedly solving the same architectural problems from scratch, software engineers use proven design templates known as design patterns.

These patterns improve:

- scalability,
- maintainability,
- flexibility,
- loose coupling,
- and software reusability .

Modern enterprise systems and frameworks such as Spring Framework, Hibernate, Android Architecture, Java Collections Framework, and GUI systems heavily rely on design patterns internally.

This question mainly focuses on:

-  **Singleton Pattern**
-  **Iterator Pattern**
-  **Command Pattern**

These patterns are extremely important for software architecture, enterprise application development, system design interviews, and scalable object-oriented programming .

QUESTION 5(a)

 **Explain Singleton Design Pattern with suitable example.**

 Answer

The **Singleton Design Pattern** is one of the most important creational design patterns in Object-Oriented Programming .

The main objective of the Singleton Pattern is to ensure that **only ONE object** of a class exists throughout the entire application. It also provides a global access point to that single object.

Why Singleton Pattern is Needed

In many real-world applications, creating multiple objects may cause resource wastage, inconsistent behavior, synchronization problems, or security issues ⚠️.

Examples include:

- database connections,
- printer spoolers,
- logging systems,
- configuration managers,
- cache managers.

Such systems require only one shared instance. The Singleton Pattern solves this problem elegantly 🚀.

🌟 Core Principles of Singleton Pattern

Singleton Pattern follows three major rules:

1. Constructor must be private.
2. Class must create its own object internally.
3. Public method must provide global access.

🌟 Real-world Analogy

Consider a country's Prime Minister 🏛️. At a given time, only one Prime Minister exists. Everyone accesses the same central authority. Similarly, the Singleton Pattern ensures only one object exists globally.

💻 Java Program for Singleton Pattern

```
class Singleton {
    private static Singleton obj = new Singleton();

    private Singleton() {
        System.out.println("Singleton Object Created");
    }

    static Singleton getInstance() {
        return obj;
    }
}

class Main {
    public static void main(String args[]) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        System.out.println(s1 == s2);
    }
}
```

💻 Output

```
Singleton Object Created  
true
```

☀ Deep Conceptual Analysis

- The constructor `private Singleton()` prevents external object creation.
- The static object `private static Singleton obj` ensures only one instance exists.
- The method `getInstance()` returns the same object repeatedly.
- Thus, `s1 == s2` returns `true`.

☀ Advantages & Disadvantages

- **Advantages:** Provides controlled object creation, memory efficiency, centralized access, and resource sharing.
- **Disadvantages:** May reduce testability, increase global dependency, and complicate multithreading if improperly implemented.

☀ Real-world Applications

Singleton Pattern is widely used in Logger classes, the `Runtime` class, Configuration managers, Database connection pools, and Cache systems.

🎯 Conclusion

Singleton Pattern ensures controlled creation of exactly one object and provides a centralized global access mechanism 🚀.

📄 QUESTION 5(b)

? **Explain Iterator Design Pattern with suitable example.**

✅ Answer

The **Iterator Design Pattern** is a behavioral design pattern that provides a systematic way to access elements of a collection sequentially without exposing internal implementation 🚀. It separates traversal logic from collection structure.

☀ Why Iterator Pattern is Needed

Suppose a collection stores books, students, products, or database records. Directly exposing the internal structure creates tight coupling, security issues, and maintenance difficulties ⚠. The Iterator Pattern solves this by providing a controlled traversal mechanism.

☀ Real-world Analogy

Consider a TV remote 📺. Users browse channels sequentially without knowing the internal wiring of the television. Iterator behaves similarly.

☀ Components of Iterator Pattern

Component	Purpose
Iterator	Traversal interface
Concrete Iterator	Actual traversal logic
Aggregate	Collection interface
Concrete Aggregate	Actual collection

💻 Java Program for Iterator Pattern

```
import java.util.*;

class Main {
    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");

        Iterator<String> it = list.iterator();

        while(it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

💻 Output

```
Java
Python
C++
```

☀ Deep Conceptual Analysis

- The method `iterator()` creates the iterator object.
- The method `hasNext()` checks whether additional elements exist.
- The method `next()` returns the next element sequentially.

Thus traversal becomes independent of the collection implementation 🚀 .

☀ Advantages & Applications

- **Advantages:** Provides clean traversal, encapsulation, reduced coupling, and flexible iteration.

- **Applications:** Used in Java Collections Framework, database cursors, file systems, tree traversal systems, and GUI components.

Conclusion

Iterator Pattern provides systematic sequential access to collection elements while hiding internal implementation details 🚀.

QUESTION 5(c)

? **Explain Command Design Pattern with suitable example.**

✓ Answer

The **Command Design Pattern** is a behavioral design pattern that converts requests into independent command objects 🚀. It encapsulates method calls, parameters, and actions inside separate command objects. This enables loose coupling, undo operations, queue processing, and request management.

☀ Why Command Pattern is Needed

Without the Command Pattern, the sender directly invokes receiver methods. This creates tight coupling, difficult extensibility, and poor flexibility ⚠. Command Pattern separates the request sender from the request executor.

☀ Real-world Analogy

Consider a restaurant 🍽. The customer gives an order to the waiter. The waiter forwards the command to the chef. The customer does not directly communicate with the kitchen. Similarly, the Command Pattern decouples the sender from the receiver 🚀.

☀ Components of Command Pattern

Component	Purpose
Command	Declares <code>execute()</code>
Concrete Command	Implements command
Receiver	Performs action
Invoker	Triggers command
Client	Creates commands

Java Program for Command Pattern

```
interface Command {  
    void execute();  
}  
  
class Light {
```

```
        void on() {
            System.out.println("Light ON");
        }
    }

    class LightCommand implements Command {
        Light l;

        LightCommand(Light l) {
            this.l = l;
        }

        public void execute() {
            l.on();
        }
    }

    class Switch {
        Command c;

        Switch(Command c) {
            this.c = c;
        }

        void press() {
            c.execute();
        }
    }

    class Main {
        public static void main(String args[]) {
            Light l = new Light();
            Command c = new LightCommand(l);
            Switch s = new Switch(c);

            s.press();
        }
    }
}
```

Output

```
Light ON
```

💡 Deep Conceptual Analysis

- The interface `Command` abstracts action execution.
- The class `LightCommand` stores request details.
- The class `Switch` acts as the invoker.

Thus sender and receiver remain loosely coupled 🚀.

☀ Advantages & Applications

- **Advantages:** Provides loose coupling, undo functionality, request queuing, logging support, and extensibility.
- **Applications:** Widely used in GUI buttons, menu systems, transaction systems, task schedulers, and remote controls.

🎯 Conclusion

Command Pattern encapsulates requests into objects and enables flexible, decoupled, and extensible request processing 🚀.

📄 QUESTION 5(d) — OR PART

? Compare Singleton, Iterator, and Command Design Patterns.

✅ Answer

Singleton, Iterator, and Command Patterns belong to different categories of design patterns and solve different software design problems 🚀.

☀ Comparison Table

Feature	Singleton	Iterator	Command
Category	Creational	Behavioral	Behavioral
Main Purpose	Single object creation	Sequential traversal	Encapsulate requests
Object Creation	Controlled	Not primary focus	Command objects
Coupling	Reduces multiple instances	Hides collection structure	Decouples sender/receiver
Real-world Example	Database Manager	Collection Traversal	Remote Control

☀ Deep Conceptual Analysis

- **Singleton** focuses on controlled object creation.
- **Iterator** focuses on collection traversal.
- **Command** focuses on request encapsulation.

Thus each pattern solves different architectural problems 🚀.

☀ Real-world Intuition

Pattern	Real-world Analogy
Singleton	Prime Minister
Iterator	TV Remote

Pattern	Real-world Analogy
---------	--------------------

Command	Restaurant Waiter
----------------	-------------------

Conclusion

Singleton, Iterator, and Command patterns solve different object-oriented design challenges and collectively improve software flexibility, maintainability, and scalability 🚀.